

RescueQR Project Overview

Direct Emergency Medical & Contact Routing System powered by Gemini AI

1. Core Concept & Working Flow

RescueQR is a web-based emergency response platform that enables first responders to immediately access a patient's vital medical history and reach their emergency contacts without friction.

The system works in the following sequential steps:

- **Scan Event:** Responders scan a physical QR code (embedded on cards, stickers, or wristbands) using any smartphone camera. This routes them to a secure details page at `/scan/[id]` without requiring an app installation.
- **GPS Capture:** Upon loading, the application requests the responder's GPS permission. If granted, the exact coordinates are captured and logged to the backend.
- **Automated Alerts:** Simultaneously, the system dispatches automated SMS and email alerts directly to the user's configured emergency contacts, providing a Google Maps location link.
- **Direct Emergency Dialing:** Responders can tap a single 'Call' button next to any contact, which triggers the phone's native calling interface (using `tel:` routing) to establish immediate, zero-friction communication.
- **AI-Powered Briefs:** The profile details are parsed by a serverless Gemini AI model to construct highly readable summaries, caregiver guidance checklists, doctor triage notes, and preparedness ratings.

2. Programming Languages & Tech Stack

The application is built with a modern, type-safe stack designed for high reliability and rapid page loading:

- **Languages:** TypeScript (core logic), JavaScript, HTML5 (semantic layout), CSS3.
- **Framework:** Next.js 15 (React 19, App Router) for serverless route handling and fast Server-Side Rendering (SSR).
- **Styling:** Tailwind CSS for utility-first styling and custom CSS tokens.
- **Database:** Supabase (PostgreSQL) with Row-Level Security (RLS) rules to guarantee medical record privacy. Includes a local storage fallback database for local offline mode.
- **AI Models:** Google's **Gemini 2.5 Flash** (via Google Language API or OpenRouter API completions gateway).

3. Backend & API Route Reference

The Next.js backend leverages serverless API Route Handlers under `/src/app/api/`. It includes intelligent API routing to support both Google Gemini Keys and OpenRouter Keys:

Endpoint	Method	Function & Details
<code>`/api/scan`</code>	POST	Logs the QR scan event, stores GPS coordinates, and triggers automated SMS/email alerts to family contacts.
<code>`/api/ai/chat`</code>	POST	Powers the floating chatbot on the landing page. Resolves user questions about setup, safety, and pricing. Accepts custom key headers.
<code>`/api/ai/summary`</code>	POST	Calls Gemini AI to synthesize a scannable 10-second emergency brief for first responders.
<code>`/api/ai/assistant`</code>	POST	Converts raw patient records into professional doctor/nurse triage notes.
<code>`/api/ai/insights`</code>	POST	Evaluates the emergency profile to calculate preparedness scores and flag allergen/medication conflicts.

4. Flexible API Key Router Mechanism

To maximize flexibility, the system features a custom, automated router. It parses the configured API key (from ``.env`` or client headers):

- **Google Gemini API Router:** If the key starts with `AIzaSy`, the backend formats requests for the Google Generative Language endpoints.
- **OpenRouter API Router:** If the key starts with `sk-` or `sk-or-`, the backend automatically reformats the payload to match the standard chat completions payload (messages array) and sends it securely to OpenRouter completions (``https://openrouter.ai/api/v1/chat/completions``).